

ASSIGNMENT # 3

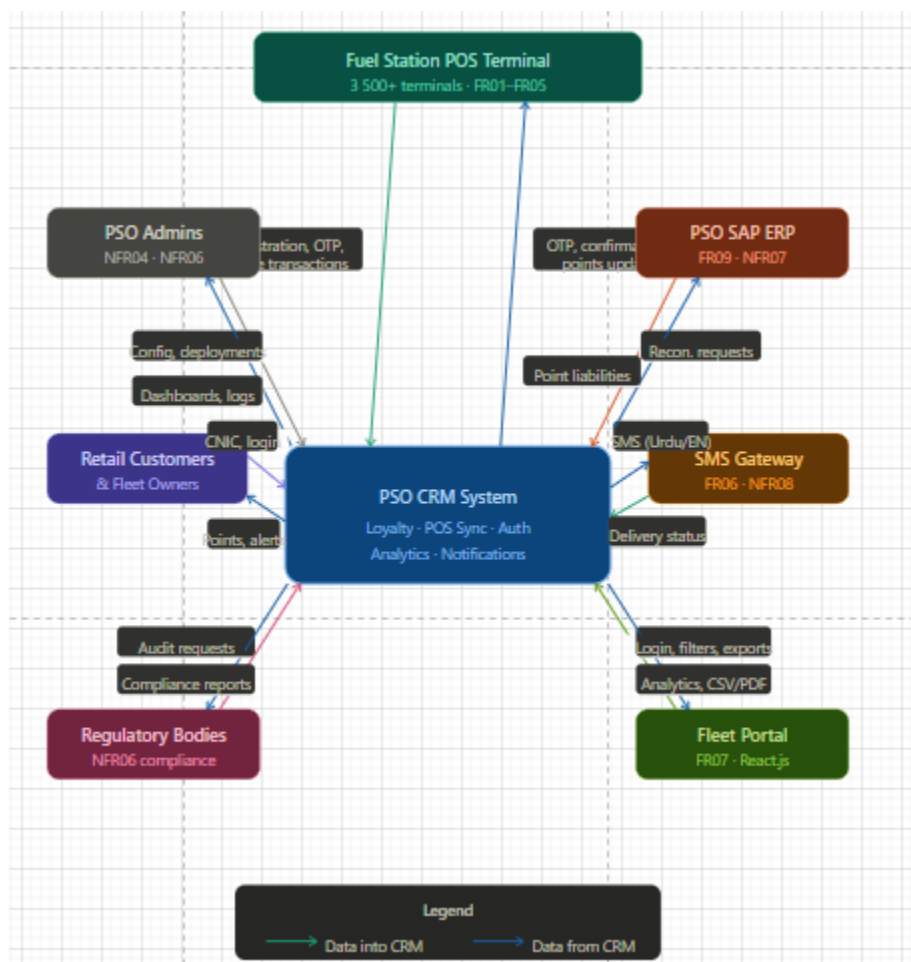
Post-Architecture Effort Estimation & Project Execution

33. PSO – CRM System | Pakistan State Oil Company Limited

BSSE23058 – Zunaira Abdul Aziz | BSSE23026 – Eashal Yasin | BSSE23003 – Fatima Noorulain | BSSE23104 – Fatima Khalid

Task (a)

1. Context diagram of the system.



1.1 Data Flows

	Entity	Data IN TO CRM	Data FROM CRM
1	POS Terminal at Stations	Customer registration data Offline transaction queues Batch sync packets OTP responses	OTP messages Transaction confirmations Sync receipts Points updates
2	Retail Customers and Fleet Owners	CNIC searches, OTP input, registration information, portal login credentials	Alerts via SMS, points balance, fleet dashboards, loyalty tier updates
3	PSO SAP ERP System	Daily requests for reconciliation, account ids, conversion rates	Daily point liabilities reconciliation flags sync timestamps
4	SMS Gateway	Delivery status reports, error callback	SMS Outbound in Urdu/English (Alerts, OTPs, Promos)
5	PSO Admins	System configuration, access updates, deployment triggers, monitoring queries	Health Dashboards Audit Logs Error Reports KPI Metrics
6	Regulatory Bodies	Compliance audit requests, changes in regulations, data policies	Compliance, audit trails, governance docs reports
7	Fleet Portal	Fleet manager login, date range filters, export requests	Fuel data, vehicle analytics, cost breakdowns, CSV/PDF exports

2. Architectural Pattern

2.1 Why We Chose Microservices

We settled on a Microservices Architecture for this system. Looking at the requirements, we had some pretty specific constraints that pushed us toward this choice. The system needs to support over 3,500 fuel station terminals scattered across the country. It has to work offline – which means when a station loses internet, transactions shouldn't just fail. We also need to scale the loyalty calculation service independently during peak hours without scaling everything else. On top of that, the requirement that the Loyalty Engine must be deployable on its own without taking down the whole system basically screams microservices.

We will break the system into smaller independent systems. Each service will do a specific business task for example calculating loyalty points. They will talk through api and message queries. If one part of the system will have an issue the rest will still be working fine.

2.2 Why This Actually Makes Sense for PSO

- **Independent Deployments:** We have to deploy the Loyalty Engine without changing anything else. In a system, changing the point calculation logic would mean we have to deploy the entire application again. With microservices, we can update just that single service on a schedule.
- **Scaling:** During morning hours, we will have a traffic spike thousands of transactions hitting points calculation at a time. With the Kubernetes orchestration, we can automatically launch more instances of just the Loyalty Engine service. We don't need to scale the entire system which saves on infrastructure costs.
- **Different Tech:** We're using Java for the backend, C/C++ for the legacy POS sync agent, C#.NET for the Fleet Portal, and JavaScript for the notification service. Microservices will let each team use the best tool for their specific job without forcing a us to adapt single technology stack on everything we have in the system.
- **Containing Failures:** If the SMS notification service stops working customers donot get their balance updates immediately but they can still buy fuel and earn points. The POS and points calculation keep working in the system. In a single system, this failure could take down the whole application which would be really bad for us.

2.3 How We are Organizing the System

2.3.1 Frontend (What Users Actually See)

We have four interfaces on client side:

- POS Terminal App uses C/C++: Runs on the 3,500+ terminals. Handles customer registration, transaction input, OTP entry, and stores every transactions locally if the internet is down.
- Fleet Portal uses React.js: A web dashboard where fleet managers can see fuel analytices, generate reports, and manage their accounts.
- Customer Mobile App (Future): We're planning to make a React Native app for customers to check their points balance and status, but that will be in phase 2.
- Admin Dashboard: Tool for IT staff to do health checks, and access audit logs.

2.3.2 The API Gateway

All client requests will go through one entry point in the API Gateway we're looking at Kong or AWS API Gateway. It is basically a traffic controller that will do several jobs

such as routes requests to the right microservice, checks authentication using JWT tokens, enforces rate limits to prevent someone from spamming our system, handles TLS encryption, and distributes load across multiple copies of each service.

2.3.3 The Core Services

Service	What It will do	Tech Stack Used	Requirements
Customer Service	Register a customer, do profile updates, do cnic validation, tier management	Java	FR01, FR04, FR08
Loyalty Engine	Calculation points, multiplier logic, redemption checks, generate otp.	Java	FR04, FR05, NFR09
POS Sync Service	Batch ingest, validation, deduplication, queue management (offline)	Java	FR02, FR03, NFR01
SAP Integration	Daily point liability export to SAP ERP, financials reconciliation	Java	FR09, NFR07
Notification Service	SMS sending through gateway, template rendering in Urdu/English	Node.js	FR06, NFR08
Analytics Service	Aggregate dashboard data, create reports, export CSV, PDF	Python or FastAPI	FR07
Auth Service	JWT issuing, session handling, audit logging, role based access	Java	NFR04, NFR06

2.3.4 Message Queue (Asynchronous Communication)

The Notification Service and SAP Integration need not wait for synchronous replies. We are using Apache Kafka as a message broker so that these services can process

events in an asynchronous manner. This decoupled process is important in the case of slow SMS delivery, it will not delay POS transactions. Kafka also gives a full history of all the events flowing through the system which is helpful for compliance and auditing.

2.3.5 Data Storage (The Database Layer)

Each microservice has a database. This is important in terms of independence one service doesn't have to wait for the database transactions of another service to complete. This is what we are using:

Customer and Loyalty (PostgreSQL) Engine services: 1. We need ACID guarantees here, because points are money. The transaction log must be durable.

Apache Cassandra for POS Sync: It is a wide-column NoSQL database to deal with massive volumes of transaction data from thousands of terminals simultaneously. Relational databases wouldn't scale to this volume and velocity.

Elasticsearch for Analytics. We want to build fleet dashboards and need to do complex aggregations across huge datasets. This is exactly what the elasticsearch inverted index is for.

Redis for Auth Service: We are caching tokens in memory to achieve sub millisecond JWT validation.

SQLite on each POS terminal: Since terminals will work offline, they need a local database for storing data until they can sync it with cloud. It is lightweight so it will run on old hardware with very little RAM needed.

2.3.6 Infrastructure & Operations

Manages all our containers and auto-scales based on traffic patterns.

- Jenkins + ArgoCD: Our CI/CD pipeline giving us blue green deployments for zero downtime updates.
- Istio Service Mesh: Encryption between services and that can automatically retry logic if a service goes down for sometime.
- Prometheus + Grafana: Monitoring dashboards which will alert us to detect any issues before it could affect our customers

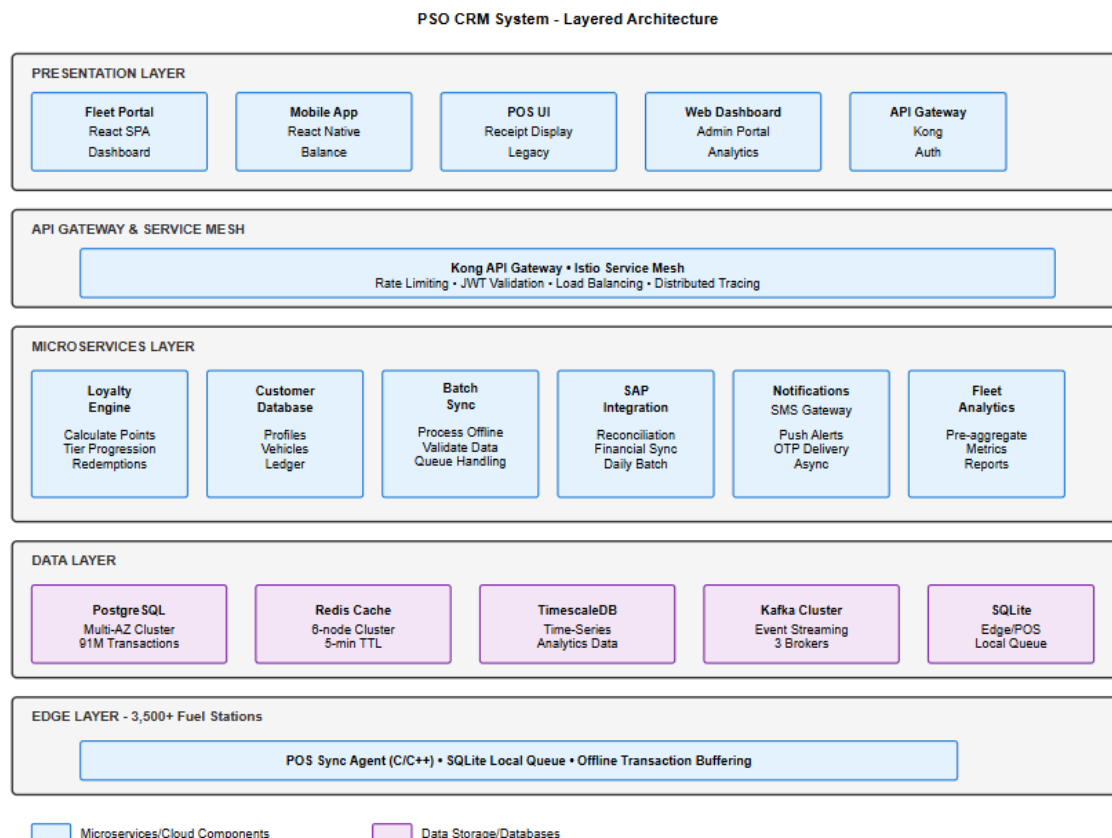
2.4 Walking Through a Real Transaction

- Customer gets 10 liters of petrol at a PSO station. The amount is captured by terminal.
- If online: POS makes an HTTP POST to the API Gateway. Offline: A transaction is kept locally.

- API Gateway validates the POS terminal credentials (JWT), implements rate-limit on the request and forwards to the POS Sync Service.
- POS Sync Service validates the transaction, checks for duplicates, stores in Cassandra and publishes a 'transaction.received' event to Kafka.
- Loyalty Engine listens on Kafka, picks up the event, gets the customer tier from PostgreSQL, calculates the points, updates the balance and publishes a 'points.awarded' event.
- Notification Service catches the 'points.awarded' event and sends SMS in the customer's choice of language
- In that same evening the SAP Integration pulls all the day's point data, calculates financial liability and syncs to PSO's ERP system.

2.5 Real Benefits of This Approach

1. Scalability: Nationwide fuel promotion, and points go crazy? Only the Loyalty Engine scales, not the POS terminal sync, not the auth service. That cuts costs.
2. Maintainability: A bug in the SMS notification service can be fixed and deployed in minutes without taking down the entire system. Different teams are able to work on different services independently.
3. Security: Everywhere across the infrastructure encryption is necessary. The Auth Service decides who can access what, meaning SAP data is only accessible to the Integration Bridge service.



3. Design Pattern.

3.1 Design Pattern Definition

A Design Pattern is a general and reusable solution to a commonly occurring problem in the context of software design. Contrary to architectural patterns that determine the overall structure of the whole system, design patterns define how classes and objects collaborate to address a certain design issue without being tied to any particular implementation. The design patterns concept was first introduced by Gamma et al. (1994) and can be classified into three categories based on their objectives:

- Creational Design Patterns – focuses on object creation processes (e.g., Singleton, Factory, Builder)
- Structural Design Patterns – deals with the composition of classes and objects (e.g., Adapter, Facade, Decorator, Proxy)
- Behavioral Design Patterns – concerns itself with algorithms and assignments of tasks to objects (e.g., Observer, Strategy, Command, Template Method)

In the PSO CRM application, several design patterns are used concurrently in various microservices and modules to solve unique problems generated by the system's FRs (FR01–FR10), NFRs (NFR01–NFR10), and the chosen Microservices Architectural Pattern in Assignment 3 Task (a) – Point 2.

3.2 Design Patterns Applied in the PSO CRM System

The following seven design patterns are applied across the PSO CRM system. Each pattern is justified by a specific design challenge arising from the project's requirements and architectural constraints.

#	Pattern	Applied In	Justification
1	Repository Pattern	Customer Service, Loyalty Engine, POS Sync Service	Database access for each microservice goes through the repository interface. Every microservice connects to its own database using the repository class, which allows separation between business logic and database access.

#	Pattern	Applied In	Justification
2	Observer / Event-Driven Pattern	Loyalty Engine, Notification Service (via Kafka)	Points.awarded messages are published by the Loyalty Engine to the Kafka topic. The Notification Service listens to these events and sends SMS to the customer without the Loyalty Engine having any idea of the SMS system. Producer/consumer decoupling
3	Strategy Pattern	Loyalty Engine – Points Calculation	Various customer levels (Silver, Gold, Platinum) have varying point multiplier calculation mechanisms. Each multiplier calculation mechanism is defined using the Strategy design pattern, where each is encapsulated inside a class implementing the PointsCalculationStrategy interface. This ensures dynamic changes to the multiplier mechanism without branching code.
4	Facade Pattern	API Gateway → Microservices	The API Gateway provides a single REST API to POS applications/web portals. This internally redirects the request to the correct micro-service (Customer/Loyalty/Sync/Analytics).
5	Adapter Pattern	SAP Integration Bridge (FR09)	The SAP ERP application makes use of the custom SAP JCo SDK interface that is not compatible with the internal REST API contract of the CRM application. The SAP Adapter class is responsible for translating DigiCash point liability objects into BAPI function call syntax for compatibility with the legacy ERP application.
6	Decorator Pattern	POS Offline Agent – Transaction Processing	Optional decorators wrap the core transactions processing logic: EncryptionDecorator (AES-256), CompressionDecorator (protobuf),

#	Pattern	Applied In	Justification
			and LoggingDecorator. Additional decorators can be added to or removed from the core transaction without altering its basic class.
7	Singleton Pattern	Database Connection Pools (all microservices)	The single instance of the connection pool per microservice exists (PostgreSQL HikariCP, Cassandra Session). The Singleton design pattern prevents unnecessary duplication of connection pools in every request, thus conserving memory resources in the light of the NFR05 requirement of using 512MB of RAM.

3.3 Primary Design Pattern – Strategy Pattern (Deep Dive)

All seven of the above-mentioned patterns can be applied to the PSO CRM project; however, the Strategy pattern stands out to be used first in designing the system since it directly relates to the main business logic, namely the calculation of DigiCash loyalty points.

3.3.1 Problem Addressed

The PSO DigiCash loyalty program has several customer tier levels (Silver, Gold, Platinum). The multipliers for the respective tiers on top of the customer fuel purchase will be different. In the naive implementation, the conditional statements (if...else/switch...case) inside the Loyalty Engine will choose which multiplier is to be applied based on the customer tier:

- Silver Tier: Multiplier is 1X (1 point/ Rs. 100)
- Gold Tier: Multiplier is 1.5X (1.5 points/ Rs. 100)
- Platinum Tier: Multiplier is 2X (2 points/ Rs. 100)

Using the conditional logic will violate OCP – adding a new tier and changing multiplier values will require the Loyalty Engine core to change, leading to possible defects due to regression testing issues since there are 3500+ POS systems using Loyalty Engine.

3.3.2 Strategy Pattern Solution

Strategy Pattern solves this problem by encapsulating the calculation behavior of each tier in separate interchangeable strategy objects that implement a common `PointsCalculationStrategy` interface:

- `PointsCalculationStrategy` Interface → Method:
`calculatePoints(purchaseAmountPKR: double): int`
- `SilverTierStrategy` implements `PointsCalculationStrategy` → $(\text{purchaseAmount} / 100) * 1$
- `GoldTierStrategy` implements `PointsCalculationStrategy` → $(\text{purchaseAmount} / 100) * 1.5$
- `PlatinumTierStrategy` implements `PointsCalculationStrategy` → $(\text{purchaseAmount} / 100) * 2$
- `LoyaltyEngine` Context Class → Stores the current strategy and invokes its `calculatePoints()` method at runtime using the tier data obtained from the database.

If the PSO marketing team later decides to introduce a new tier and alter the multiplier factor, all they need to do is write a new strategy object; the `LoyaltyEngine` class doesn't have to be altered at all.

3.3.3 How the Strategy Pattern Integrates with the Microservices Architecture

Regarding the selected Microservices Architecture for the PSO CRM:

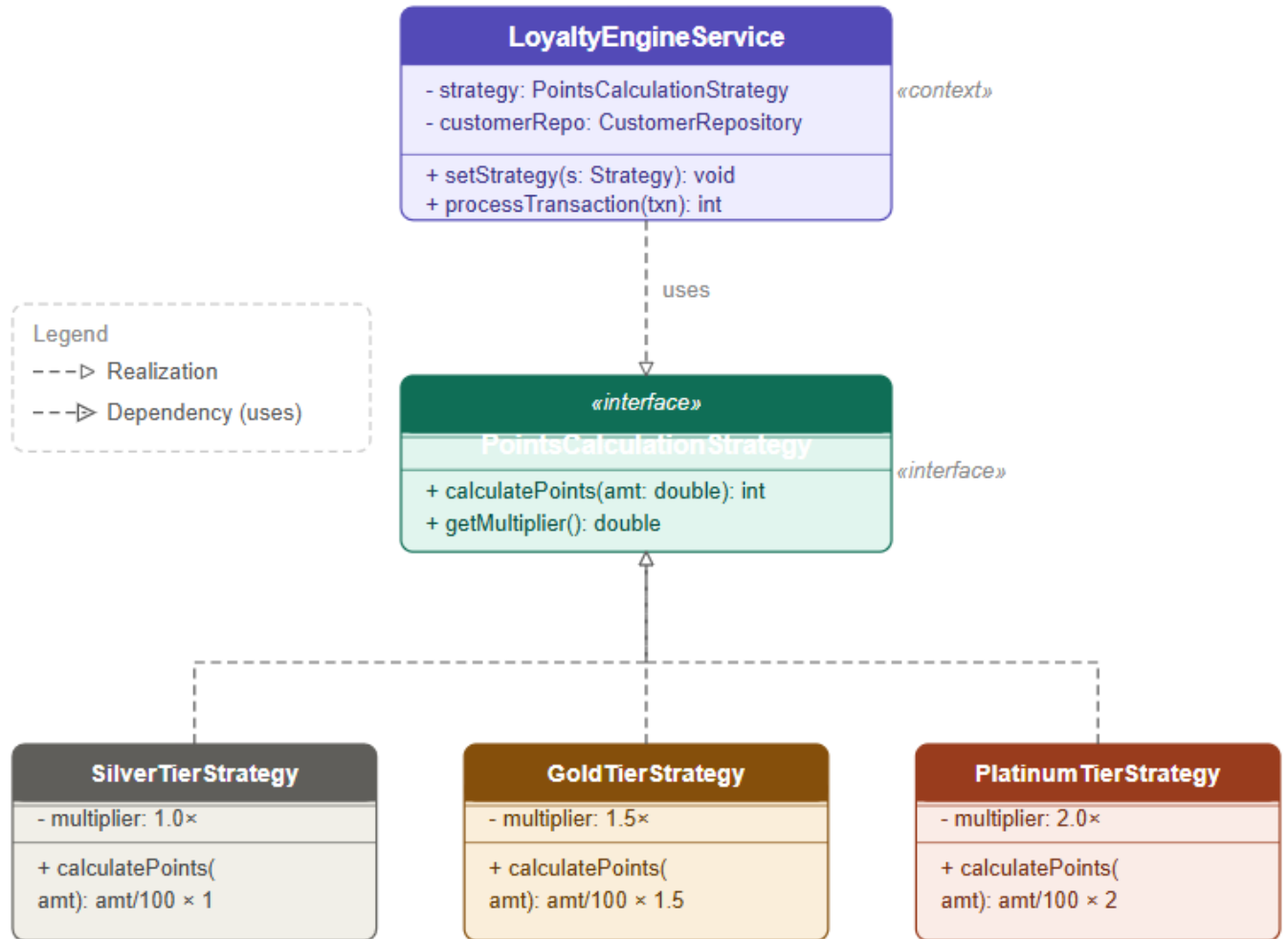
- The `LoyaltyEngineService` microservice runs as a separate Kubernetes pod.
- After receiving a `points.calculate` Kafka message (from the POS Sync Service), the Loyalty Engine fetches the tier information of the customer from its PostgreSQL database.
- Then the proper `PointsCalculationStrategy` object will be created using the `StrategyFactory` class.
- The points will be calculated and the customer balance will be updated by Loyalty Engine in PostgreSQL points ledger transactionally.
- The `points.awarded` event will be posted on Kafka, which will be processed by Notification Service for sending the SMS notification.

This architectural solution provides isolation of tier multiplier changes via Loyalty Engine's blue-green deployment (NFR09) without influencing any other microservices, thus meeting requirements for the Strategy pattern usage and zero downtime deployment.

3.4 Summary — Design Pattern Mapping to Requirements

Pattern	Category	FR / NFR Addressed	Key Benefit
Repository	Structural	FR01, FR02, FR03, FR04, FR08	Decouples business logic from data store; simplifies unit testing
Observer / Event-Driven	Behavioural	FR04, FR06, NFR09	Decouples Loyalty Engine from Notification Service via Kafka
Strategy	Behavioural	FR04, FR05, NFR09	Swappable tier multiplier logic; satisfies OCP; zero-downtime updates
Facade	Structural	FR01–FR10, NFR04	Single API Gateway entry point; hides microservice topology from clients
Adapter	Structural	FR09, NFR07	Bridges incompatible SAP JCo SDK with CRM REST API contract
Decorator	Structural	FR02, FR03, NFR04, NFR05, NFR10	Composable encryption + compression + logging on POS transactions
Singleton	Creational	NFR05, NFR02	Single connection pool per service; preserves 512 MB RAM on POS agent

Strategy Pattern – PSO CRM Loyalty Engine



4. Division of the project in various activities/features/module

4.1 Introduction to the WBS

The Work Breakdown Structure (WBS) involves a hierarchical breakdown of all the tasks needed to accomplish the project objectives. The WBS involves breaking down the project scope of work into phases, features, modules and activities, which can be assigned to team members, estimated for effort, planned for execution within sprints and tracked for completion status. The WBS is one of the primary project planning documents and is mandatory in the Project Management Body of Knowledge (PMBOK Guide). The WBS is also used in the COCOMO II estimation model.

The WBS for PSO CRM is organized around seven phases, each of which corresponds to one or more Scrum sprint, based on the PBS developed in Assignment 1 and microservices architecture developed in Assignment 3 point 2. The seven phases correspond to the seven microservices and product categories identified in PBS.

4.2 WBS Hierarchy Levels

- Level 1 – Project: PSO CRM System Implementation
- Level 2 – Phase: Seven delivery phases (Infrastructure, Customer Management, POS Module, Loyalty Engine, Integration & Analytics, Security & Compliance, Project Management)
- Level 3 – Feature / Module: Individual deliverable components within each phase (e.g., Auth Service, Batch Sync Service, Strategy Pattern Engine)
- Level 4 – Activity / Task: Specific implementation tasks (e.g., 'Design SQLite schema', 'Implement CNIC tokenisation', 'Configure Kubernetes HPA')

4.3 WBS Table — Full Decomposition

The following table presents the complete Work Breakdown Structure for the PSO CRM project, decomposed to Activity level. Story Points (SP) are carried forward from the Scrum Product Backlog in Assignment 3 Section 4. Phases are colour-coded for visual clarity.

WBS ID	Phase	Feature / Module	Activities / Tasks	Sprint	Assigned To	SP
1.0	Infrastructure & Foundation					
1.1	Environment Setup	Kubernetes Cluster, CI/CD	Install K8s cluster; configure Jenkins + ArgoCD; set up Docker	Sprint 1	DevOps Lead	5

WBS ID	Phase	Feature / Module	Activities / Tasks	Sprint	Assigned To	SP
		Pipelines, Docker Registry	registry; provision namespaces			
1.2	Database Provisioning	PostgreSQL, Cassandra, Redis, Elasticsearch	Provision DB servers; create schemas/keyspaces; configure connection pools; enable AES-256 at rest	Sprint 1	DB Engineer	5
1.3	Kafka & Istio	Message Broker & Service Mesh	Deploy Kafka cluster; define topics (txn.received, points.awarded); install Istio service mesh with mTLS	Sprint 1	DB Engineer	3
1.4	Auth Service	JWT, RBAC, Audit Logging (NFR04, NFR06)	Implement JWT issuance & validation; define RBAC roles; implement audit logging with 7-year retention	Sprint 1	Backend Eng.	5
2.0	Customer Management					
2.1	Customer Service	Registration, Profile, CNIC Validation (FR01)	Build registration API; implement CNIC uniqueness validator; persist 15 DETs; activate loyalty account	Sprint 2	Backend Eng.	5
2.2	Unified Customer DB	ILF – 15 DETs, 4 RETs (FR08)	Design & implement PostgreSQL schema; add CNIC field-level tokenisation; test 1M+ record scalability	Sprint 2	DB Engineer	8
2.3	Balance Inquiry	Real-time EQ – Read-only (FR10)	Build GET /balance?cnic= endpoint; validate CNIC; return balance + PKR equivalent in < 2.5 s	Sprint 2	Backend Eng.	3

WBS ID	Phase	Feature / Module	Activities / Tasks	Sprint	Assigned To	SP
3.0	Offline-First POS Module					
3.1	POS Offline Agent	C/C++ SQLite Queue (FR02, NFR01, NFR05)	Implement local SQLite encrypted queue; persist transactions during outage; queue survives power cycle; < 512 MB RAM	Sprint 3	POS Developer	8
3.2	Batch Sync Service	Protobuf Compression, Checksum (FR03, NFR10)	Build auto-trigger on reconnection; implement protobuf serialisation (< 20 KB/txn); checksum validation; deduplication logic	Sprint 3	Backend Eng.	8
3.3	POS UI	Registration & Transaction UI on POS Terminal	Design lightweight POS UI (NFR05 Pentium hardware); bilingual display (NFR08 Urdu/English); OTP entry screen	Sprint 3	POS Developer	3
4.0	Loyalty Engine					
4.1	Points Engine	Calculation, Tiers, Multipliers (FR04)	Implement PointsCalculationStrategy per tier; read multiplier from DB; update balance < 2.5 s; re-evaluate tier on threshold crossing	Sprint 4	Backend Eng.	5
4.2	OTP Redemption	Redemption Authorisation (FR05)	Generate & dispatch OTP via SMS; validate in 3-min window; deduct points; log redemption; update balance atomically	Sprint 4	Backend Eng.	5
4.3	Notification Service	SMS Gateway, Urdu/English (FR06, NFR08)	Subscribe to Kafka points.awarded events; render Urdu/English SMS templates; dispatch via third-party SMS gateway; handle delivery callbacks	Sprint 4	Backend Eng.	3

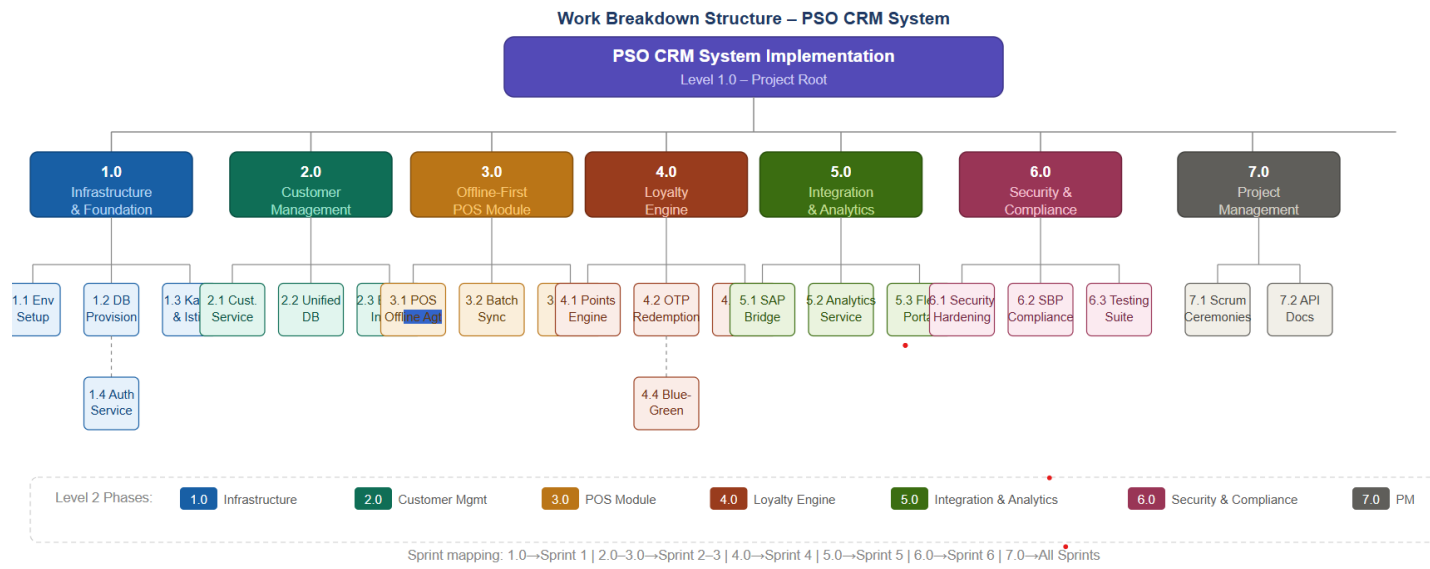
WBS ID	Phase	Feature / Module	Activities / Tasks	Sprint	Assigned To	SP
4.4	Blue-Green Deploy	Zero-Downtime Pipeline (NFR09)	Configure blue-green deployment for Loyalty Engine in ArgoCD; verify zero-downtime during switchover in staging environment	Sprint 6	DevOps Lead	5
5.0	Integration & Analytics					
5.1	SAP Integration Bridge	EIF – Financial Reconciliation (FR09, NFR07)	Implement SAP Adapter (JCo SDK); nightly job at 02:00 AM; failover to passive node; verify 99.9% uptime SLA	Sprint 5	DB Engineer	8
5.2	Analytics Service	Fleet Dashboard, Reports (FR07)	Build aggregation queries on Elasticsearch; implement date/vehicle/cost-centre filters; CSV + PDF export; dashboard load < 3 s	Sprint 5	Backend Eng.	8
5.3	Fleet Web Portal	React.js SPA for Fleet Managers	Build React.js single-page application; integrate REST API calls to Analytics Service; role-based access for fleet managers	Sprint 5	POS Developer	5
6.0	Security, Compliance & Testing					
6.1	Security Hardening	AES-256, TLS 1.3, Penetration Test (NFR04)	Audit AES-256 at rest for all DBs; enforce TLS 1.3 at API Gateway; run third-party penetration test; remediate findings	Sprint 6	DevOps Lead	5
6.2	SBP Compliance	Regulatory Reporting, Audit Trails (NFR06)	Implement 7-year audit log retention; generate SBP compliance report; SECP data governance documentation; data	Sprint 6	Backend Eng.	3

WBS ID	Phase	Feature / Module	Activities / Tasks	Sprint	Assigned To	SP
			retention policy enforcement			
6.3	Testing Suite	Unit, Integration, Load, UAT	Write JUnit/Postman tests; simulate 3,500 concurrent outlets for NFR02; conduct UAT at 5 pilot stations; document results	Sprint 6	QA Lead	5
7.0	Project Management & Documentation					
7.1	Project Management	Scrum Ceremonies, Jira, Burndown Charts	Run sprint planning, daily standups, reviews, retrospectives; maintain Jira backlog; produce velocity & burndown charts	All Sprints	Scrum Master	–
7.2	Documentation	API Contracts, User Manuals, SOPs	Produce API documentation (OpenAPI/Swagger); write user training manuals for station staff; create Standard Operating Procedures for POS sync	Sprint 5–6	All Team	–

4.4 WBS Summary by Phase

Phase	Phase Name	Modules / Features	Sprint(s)	Assigned To	Total SP
1.0	Infrastructure & Foundation	Env Setup, DB Provisioning, Kafka/Istio, Auth Service	Sprint 1	All Team	18
2.0	Customer Management	Customer Service, Unified DB, Balance Inquiry	Sprint 2	Backend + DB Eng.	16

Phase	Phase Name	Modules / Features	Sprint(s)	Assigned To	Total SP
3.0	Offline-First POS Module	POS Offline Agent, Batch Sync, POS UI	Sprint 3	POS Dev + Backend Eng.	19
4.0	Loyalty Engine	Points Engine, OTP Redemption, Notification, Blue-Green	Sprint 4 + 6	Backend Eng. + DevOps	18
5.0	Integration & Analytics	SAP Bridge, Analytics Service, Fleet Web Portal	Sprint 5	DB Eng. + Backend + POS Dev	21
6.0	Security, Compliance & Testing	Security Hardening, SBP Compliance, Testing Suite	Sprint 6	DevOps + Backend + QA	13
7.0	Project Management & Documentation	Scrum Ceremonies, Jira, API Docs, User Manuals	All Sprints	All Team	–
TOTAL STORY POINTS (Phases 1–6)					105



4.5 WBS Notes & Assumptions

- Estimation is based on story points following the Fibonacci sequence (1, 2, 3, 5, 8, 13) in line with the Scrum Product Backlog in Assignment 3 Section 4.3.
- Phase 7 (Project Management & Documentation) is an overlay phase for management overhead and does not have any story points assigned as it runs throughout all sprints in parallel.
- The WBS is tied to the six 2-week sprints specified in the Scrum sprint roadmap. Sprint 3 and Sprint 5 have the highest story points (19 and 21 SP respectively) owing to the inherent difficulty involved in implementing the offline queuing mechanism and integrating SAP.
- The DoD criteria for all WBS activities are implicitly included: code review and merge into the main branch, successful unit testing in the CI pipeline, product owner validation of acceptance criteria, and deployment to the staging environment.
- The base SLOC count of 2,250 SLOC determined in Assignment 2 and maintained in the Post-Architecture COCOMO II model in Assignment 3 Point 3 forms the basis of effort estimation for all WBS activities. Phase 3 (POS Module) and Phase 5 (SAP Integration) are the major contributors to the CPLX High cost driver in Section 3.3 of the COCOMO II model.

4.6 Traceability — WBS to Requirements

The following table demonstrates the traceability from WBS phases to the functional and non-functional requirements established in Assignment 2, confirming that all requirements are covered by at least one WBS work package.

Phase	Phase Name	FR Covered	NFR Covered
1.0	Infrastructure & Foundation	FR08 (partial), FR09 (partial)	NFR02 (Kubernetes HPA), NFR04 (AES-256 DB setup), NFR06 (Audit Log), NFR07 (Active-Passive Failover)
2.0	Customer Management	FR01, FR08, FR10	NFR03 (< 2.5 s balance inquiry), NFR04 (CNIC tokenisation)
3.0	Offline-First POS Module	FR02, FR03	NFR01 (72-hr offline retention), NFR05 (512 MB RAM), NFR08 (Urdu/English POS UI), NFR10 (< 20 KB protobuf)
4.0	Loyalty Engine	FR04, FR05, FR06	NFR03 (< 2.5 s points update), NFR08 (Urdu/English SMS), NFR09 (Blue-Green Deploy)
5.0	Integration & Analytics	FR07, FR09	NFR07 (99.9% SAP Uptime), NFR02 (Fleet Dashboard Scalability)
6.0	Security, Compliance & Testing	All FR01–FR10 (UAT coverage)	NFR02 (Load Test 3,500 outlets), NFR04 (TLS 1.3, Penetration Test), NFR06 (SBP/SECP Compliance)

5. Definition of the object classes, with estimated procedures and attributes.

5.1 Introduction

The PSO CRM system is built using object-oriented design and microservices. Each microservice includes a set of classes and services. These classes and services are based on patterns. The patterns used are Repository Pattern, Strategy Pattern and Event-Driven architecture. This section explains the classes, their important details and procedures.

Procedures are estimated based on complexity. The complexity is measured as Low, Medium or High. This estimation is done using COCOMO II assumptions and WBS story points. The PSO CRM system uses these classes and procedures. They are designed to work. The microservices in the PSO CRM system are separate. Each one handles a task. The PSO CRM system is designed to be flexible. It can be easily. Changed. The classes and procedures, in the PSO CRM system are the building blocks.

They help the system work correctly.

5.2 Core Domain Object Classes

1. Customer Class

Purpose: Represents a PSO customer registered in the DigiCash loyalty system.

Attributes:

- customerId: UUID
- cnic: String (encrypted/tokenized)
- name: String
- phoneNumber: String
- email: String
- tierLevel: Enum (SILVER, GOLD, PLATINUM)
- registrationDate: Date
- status: Boolean (active/inactive)

Procedures (Methods):

- registerCustomer()
- updateProfile()
- validateCNIC()
- upgradeTier()
- deactivateAccount()

Estimated Complexity: Medium ($\approx$ 300–400 SLOC equivalent)

2. LoyaltyAccount Class

Purpose: Maintains loyalty points balance and transaction history reference.

Attributes:

- accountId: UUID
- customerId: UUID
- totalPoints: int
- redeemedPoints: int
- availablePoints: int
- lastUpdated: Timestamp

Procedures:

- addPoints(int points)
- redeemPoints(int points)
- getBalance()
- updateLedger()
- calculateAvailablePoints()

Estimated Complexity: Medium-High ($\approx$ 350–500 SLOC)

3. Transaction Class

Purpose: Represents a fuel purchase transaction from POS terminals.

Attributes:

- transactionId: UUID
- customerId: UUID
- stationId: String
- fuelAmountPKR: double
- fuelLitres: double
- timestamp: DateTime
- syncStatus: Enum (PENDING, SYNCED, FAILED)
- encryptedPayload: String

Procedures:

- createTransaction()
- validateTransaction()
- encryptPayload()
- markAsSynced()
- retrySync()

Estimated Complexity: High ($\approx$ 450–600 SLOC)

4. POSOfflineAgent Class

Purpose: Handles offline transaction storage at POS terminals.

Attributes:

- localQueue: SQLite DB reference
- maxStorageLimit: int (MB)
- connectivityStatus: Boolean
- retryCounter: int

Procedures:

- storeTransactionOffline()
- detectConnectivity()
- syncWhenOnline()
- compressQueueData()
- purgeOldRecords()

Estimated Complexity: High ($\approx$ 500–650 SLOC)

5. SyncService Class

Purpose: Synchronises offline POS transactions with central CRM system.

Attributes:

- kafkaProducer: Kafka instance
- batchSize: int
- syncInterval: int (seconds)
- checksumValidator: String utility

Procedures:

- batchSyncTransactions()
- publishToKafka()
- validateChecksum()
- resolveDuplicates()
- handleSyncFailure()

Estimated Complexity: High ($\approx$ 450–600 SLOC)

6. LoyaltyEngine Class (Context Class – Strategy Pattern)

Purpose: Core engine for calculating loyalty points.

Attributes:

- strategy: PointsCalculationStrategy
- customerTier: Enum
- calculationCache: Map

Procedures:

- calculatePoints(double amount)
- setStrategy(PointsCalculationStrategy strategy)
- fetchCustomerTier()
- updatePointsLedger()
- emitPointsEvent()

Estimated Complexity: High ($\approx$ 400–550 SLOC)

7. PointsCalculationStrategy (Interface)

Purpose: Defines contract for all tier-based point calculations.

Attributes:

- (No direct attributes – interface only)

Procedures:

- calculatePoints(double amount)

Implementations:

- SilverStrategy
- GoldStrategy
- PlatinumStrategy

Estimated Complexity: Low per class ($\approx$ 80–120 SLOC each implementation)

8. NotificationService Class

Purpose: Sends SMS/email notifications when points are awarded.

Attributes:

- smsGateway: API client
- templateEngine: String renderer
- languagePreference: Enum (URDU/ENGLISH)

Procedures:

- sendPointsNotification()
- formatMessage()
- fetchTemplate()

- handleDeliveryStatus()
- retryFailedNotifications()

Estimated Complexity: Medium ($\approx$ 300–400 SLOC)

9. SAPAdapter Class (Integration Layer)

Purpose: Connects CRM system with SAP ERP via translation layer.

Attributes:

- sapConnection: JCo interface
- mappingRules: Config file
- retryPolicy: int

Procedures:

- convertToSAPFormat()
- sendToSAP()
- fetchSAPResponse()
- handleIntegrationFailure()
- logSAPTransaction()

Estimated Complexity: High ($\approx$ 400–600 SLOC)

10. AuthService Class

Purpose: Handles authentication and authorization across system.

Attributes:

- jwtSecretKey: String
- roleMatrix: Map<Role, Permissions>
- sessionStore: Redis cache

Procedures:

- generateJWT()
- validateToken()
- assignRole()
- revokeAccess()
- logAuthenticationEvent()

Estimated Complexity: Medium ($\approx$ 300–450 SLOC)

11. AuditLog Class

Purpose: Maintains compliance-grade system activity logs.

Attributes:

- logId: UUID
- eventType: String
- timestamp: DateTime
- userId: UUID
- details: JSON

Procedures:

- createLogEntry()
- archiveLogs()
- retrieveLogs()
- enforceRetentionPolicy()
- exportAuditReport()

Estimated Complexity: Medium ($\approx$ 250–350 SLOC)

12. AnalyticsService Class

Purpose: Generates reports and fleet analytics dashboards.

Attributes:

- dataSource: Elasticsearch
- cacheLayer: Redis
- reportTemplates: JSON configs

Procedures:

- generateFleetReport()
- filterTransactions()
- aggregateData()
- exportCSV()
- renderDashboardMetrics()

Estimated Complexity: Medium-High ($\approx$ 400–550 SLOC)

5.3 Summary of Class Complexity Distribution

Class Category	Count	Complexity Level
Core Domain Classes (Customer, Transaction, LoyaltyAccount)	3	High
Service Classes (LoyaltyEngine, SyncService, NotificationService)	3	High
Integration Classes (SAPAdapter)	1	High
Support Classes (Auth, Audit, Analytics)	3	Medium
Strategy Interfaces	1	Low

5.4 Overall Design Observation

- The system follows a **clean separation of concerns**:
 - Domain Layer → Customer, Transaction, LoyaltyAccount
 - Service Layer → LoyaltyEngine, SyncService
 - Integration Layer → SAPAdapter
 - Infrastructure Layer → AuthService, AuditLog

Most of the work is, in handling sales when the system is offline figuring out loyalty rewards and getting our system to work with SAP. This is what we are doing in the tasks that need a lot of work (Sprint 3–5).

We use the Strategy Pattern. This helps us make changes to the system without messing with the parts. It makes the system easier to maintain and work with over time.

6. Post-architecture estimate of the system

In Assignment 2, we calculated the Early Design Estimate based on UFP and 7 Effort Multipliers. Now that the system architecture is clearly defined (Microservices, Strategy/Observer patterns, Kafka queues), we must recalculate the effort using the 17 Post-ArchitectureEM.

6.1 Baseline Constants (from Assignment 2)

- **Estimated Size:** 2.25 KLOC
- **COCOMO II Constant (A):** 2.94
- **Exponent (B):** 1.0957
- **Calculated Base Effort:** 7.168 Person-Months (PM)
- **Original Early Design Estimate:** 12.30 PM

6.2 Post-Architecture Effort Multipliers (EM)

We evaluated the 17 cost drivers based on our actual design decisions:

Category	Cost Driver	Rating	Multiplier	Justification
Product	RELY	High	1.10	High stakes; points represent actual financial value for PSO.
	DATA	High	1.14	We are managing massive transaction logs from 3,500+ stations.
	CPLX	High	1.17	Microservices, async syncs, and SAP integration add complexity.
	RUSE/DOCU	Nominal	1.00	Standard industry practices for reuse and documentation.
Platform	STOR	High	1.05	The 512MB RAM limit on old POS terminals is a tight constraint.
	TIME/PVOL	Nominal	1.00	Infrastructure is stable; scaling is handled by Kubernetes.

Personnel	ACAP/PCAP	High	0.85/0.88	Our team has strong core capabilities in architecture and coding.
	APEX	Low	1.10	The fuel/loyalty domain is relatively new to the dev team.
	PLEX/LTEX	Low	1.09/1.09	Learning curves for specialized tools like Kafka and SQLite C++ wrappers.
Project	TOOL/SITE/SCED	Nominal	1.00	Standard tools and schedule; team is fully colocated.

6.3 Final Recalculation

- **New Effort Adjustment Factor (EAF): 1.503**
- **Post-Architecture Effort:** $7.168 \times 1.503 = \mathbf{10.77 \text{ Person-Months}}$

6.4 Impact Analysis: What Changed?

- **Early Design Estimate:** 12.30 PM
- **Post-Architecture Estimate:** 10.77 PM
- **Net Reduction: 1.53 Person-Months**

Conclusion

Our analysis shows a reduction of 1.53 person-months compared to our initial estimate. This shift occurs because "Early Design" estimates are naturally more conservative (higher) to account for unknowns.

By defining a Microservices architecture and using the Strategy Pattern, we have effectively "modularized" the complexity. Even though we accounted for the learning

curves associated with Kafka and legacy POS hardware (LTEX/PLEX), these are outweighed by our team's high technical capability (ACAP/PCAP) and the clarity provided by our architectural choices. In short, a well-defined architecture has reduced the perceived risk and streamlined the projected effort required to deliver the PSO CRM system.

References

Boehm, B. W. et al. (2000). Software cost estimation with COCOMO II. Prentice Hall PTR.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

PMI. (2021). A guide to the project management body of knowledge (PMBOK Guide), 7th ed. Project Management Institute.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

PSX. (2024). Pakistan State Oil Company Limited – Company profile.
<https://dps.psx.com.pk/company/PSO>